

A Comprehensive Framework for Secure Multi-Agent Retrieval-Augmented Generation Systems

Chaima Ghouil^{1,2}, Radhoine Aloui³, Nejib Khalfaoui⁴, Sofien Mhatli¹, Adel Aldalbahi⁵, Ignacio Llamas Garro⁶

¹SERCOM-Lab, EPT, University of Carthage, La Marsa 2078, Tunisia

²ISI KEF, University of Jendouba, Tunisia

³Science and Technology for Defense Laboratory (STD), Military Research Center, Tunis, Tunisia

⁴Higher School of Engineers of Medjez El Bab (ESIM) 9070 Medjez el-Bab

⁵Department of Electrical Engineering, King Faisal University, Al-Ahsa, Saudi Arabia

⁶Centre Tecnològic de Telecomunicacions de Catalunya (CTTC/CERCA), Castelldefels, Spain

Corresponding authors: aaldalbahi@kfu.edu.sa, chaimaghoul8@gmail.com

Abstract—This paper presents a comprehensive framework for securing Retrieval-Augmented Generation (RAG) systems in multi-agent environments. We introduce two complementary architectures: the Secure RAG Framework (SecRAG) for layered defense mechanisms and the Secure Multi-Agent RAG Simulator (SMARS) for systematic security evaluation. Our framework addresses critical vulnerabilities including prompt injection, data poisoning, and coordinated attacks through quantitative risk assessment and mitigation strategies. Experimental results demonstrate that our hybrid defense approach reduces attack success rates by up to 89% while maintaining system performance within acceptable thresholds. The proposed framework provides a holistic solution for securing complex multi-agent RAG systems in real-world deployments.

Index Terms—Multi-Agent Systems, Retrieval-Augmented Generation, RAG Security, Cybersecurity, Adversarial Attacks, Simulation Framework

I. INTRODUCTION

The rapid advancement of Large Language Models (LLMs) has enabled sophisticated applications through Retrieval-Augmented Generation (RAG) systems, which combine parametric knowledge with external information retrieval [1], [2], [24], [28]. When deployed in multi-agent environments, these systems face amplified security challenges due to increased attack surfaces, complex interaction patterns, and coordinated adversarial actions [3], [4].

Multi-agent RAG systems are particularly vulnerable to novel attack vectors including:

- **Coordinated attacks** exploiting inter-agent communication channels [5]
- **Retrieval poisoning** affecting multiple agents simultaneously [6]
- **Propagation vulnerabilities** where compromised agents influence others
- **Distributed denial of service** targeting shared resources

Despite growing recognition of these threats [7], [8], existing security frameworks remain inadequate for multi-agent

RAG environments. Prior work has either focused on single-agent scenarios without considering inter-agent propagation effects [9], [10], or addressed multi-agent security without incorporating RAG-specific retrieval vulnerabilities [3]. Our work bridges this gap by integrating a defense-in-depth strategy with quantitative risk assessment tailored to the unique challenges of multi-agent RAG systems.

A. Contributions

This paper makes three primary contributions:

- 1) A **layered defense architecture (SecRAG)** integrating security mechanisms across the entire RAG pipeline.
- 2) A **simulation framework (SMARS)** for controlled security evaluation in multi-agent environments.
- 3) A **comprehensive risk assessment methodology** with quantitative metrics and mitigation strategies.

Unlike existing approaches that either focus on single-agent RAG security or general multi-agent system protection, this work introduces a unified framework specifically designed for multi-agent RAG environments. The novelty of our approach lies in three key aspects: (1) the integration of a defense-in-depth architecture tailored to the full RAG pipeline in multi-agent settings, (2) the design of the SMARS simulation framework enabling controlled and reproducible security evaluation, and (3) the introduction of a quantitative risk assessment model that captures both individual and coordinated attack impacts. This combination goes beyond simple integration of existing methods by providing a cohesive and extensible framework for analyzing and securing complex RAG-based systems.

B. Paper Organization

The remainder of this paper is organized as follows: Section II reviews related work; Section III presents the SecRAG architecture; Section IV describes the SMARS simulator; Section V details the risk assessment framework; Section VI presents experimental methodology; Section VII discusses results; and Section IX concludes with future directions.

II. RELATED WORK

A. Single-Agent RAG Security

Research on RAG security has primarily focused on single-agent scenarios. Greshake et al. [11] identified indirect prompt injection vulnerabilities, while Zou et al. [6] explored knowledge corruption attacks. Privacy concerns in RAG systems were examined by Zeng et al. [12], highlighting data leakage risks. Jiang et al. [13] demonstrated agent-based attacks for data extraction. However, these studies lack consideration of multi-agent interactions and propagation effects. Defense mechanisms for single-agent RAG include input sanitization [14], embedding perturbation [15], and retrieval re-ranking [16]. Evaluation frameworks like RAGAS [9] and ARES [10] provide metrics for retrieval and generation quality but lack multi-agent security assessments.

B. Multi-Agent System Security

Traditional multi-agent security research addresses communication integrity [3], [25], [27], [33], trust management, and distributed consensus. Chen et al. [17] investigated opinion manipulation in cooperative systems. Zhang et al. [4] analyzed security threats in multi-agent communication systems. However, these works do not incorporate RAG-specific vulnerabilities related to external knowledge retrieval.

C. Security Simulation Frameworks

Existing simulation platforms for AI security [18], [29] focus on training-time attacks or single-model scenarios. Jagielski et al. [19] presented poisoning attacks and countermeasures. None provide comprehensive support for multi-agent RAG systems with configurable retrieval mechanisms and inter-agent communication protocols.

D. Risk Assessment Frameworks

Risk assessment frameworks for AI systems include the NIST AI RMF [8] and OWASP guidelines [7]. Ammann et al. [20] proposed a RAG-specific risk assessment framework, but without multi-agent considerations. Our work extends these frameworks to multi-agent environments.

TABLE I: Comparison of existing approaches with proposed framework

Approach	RAG Support	Multi-Agent	Security Metrics	Simulation
Classic RAG [1]	Yes	No	Limited	No
MAS Security [3]	No	Yes	General	Partial
Attack Simulators [18]	Partial	No	Attack-focused	Yes
Our Framework	Yes	Yes	Comprehe	Yes

III. SECURE RAG FRAMEWORK (SEC-RAG)

A. Architecture Overview

The Secure RAG Framework (SecRAG) implements a defense-in-depth strategy with four security layers, as illustrated in Figure 1. Each layer addresses specific vulnerabilities

while maintaining system performance, building upon principles from [7], [20].

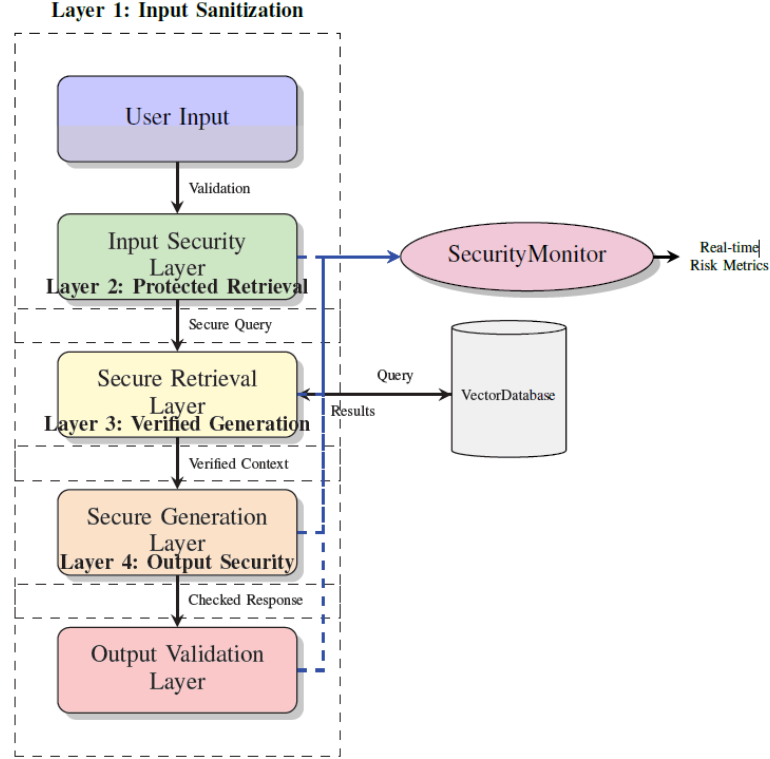


Fig. 1: SecRAG architecture with four security layers and cross-layer monitoring

B. Layer 1: Input Sanitization

This layer prevents malicious inputs from entering the system. It performs syntax validation to check input format and structure, semantic analysis to detect adversarial intent using pattern recognition, rate limiting to prevent denial-of-service attacks, and context verification to ensure query relevance to system capabilities. These measures directly address vulnerabilities identified in [11], [21], [34].

C. Layer 2: Protected Retrieval

The protected retrieval layer ensures the integrity of the retrieval process. It includes document authentication to verify source credibility and integrity, similarity thresholding to filter irrelevant or potentially malicious documents, privacy preservation via differential privacy in embeddings [26], [31], and freshness validation to ensure retrieved information is current. This layer counters attacks from [6], [18].

To enhance privacy preservation, we incorporate differential privacy mechanisms into the embedding generation process. Specifically, calibrated noise is injected into embedding vectors according to a controlled privacy budget, limiting the risk of sensitive information leakage through embedding inversion attacks. This approach ensures that small changes in the input data do not significantly affect the resulting embeddings,

thereby protecting individual data points. In our implementation, we use Gaussian noise with sensitivity bounds as a practical instantiation of differential privacy.

D. Layer 3: Verified Generation

To secure the response generation process, this layer performs context validation as a secondary verification of retrieved content, constraint enforcement to apply safety guidelines, uncertainty quantification to measure response confidence levels, and multi-model verification by cross-checking outputs with alternative models. This approach builds on [14].

E. Layer 4: Output Security

Before delivering a response, the output security layer performs factual accuracy checking against trusted knowledge sources, safety filtering to remove harmful or inappropriate content, watermarking to embed traceable identifiers [36], and quality assurance to assess response coherence and usefulness. These steps address concerns raised in [12].

F. Security Monitor

The cross-layer security monitor provides comprehensive oversight as recommended in [8]. It performs real-time threat detection through continuous monitoring of system state, dynamic risk scoring to calculate the current security posture, incident response coordination (both automated and manual), and forensic logging to maintain a comprehensive audit trail for post-hoc analysis.

IV. SECURE MULTI-AGENT RAG SIMULATOR (SMARS)

A. Simulator Architecture

SMARS provides a controlled environment for security evaluation, with architecture shown in Figure 2. The design incorporates multi-agent security principles from [3], [4].

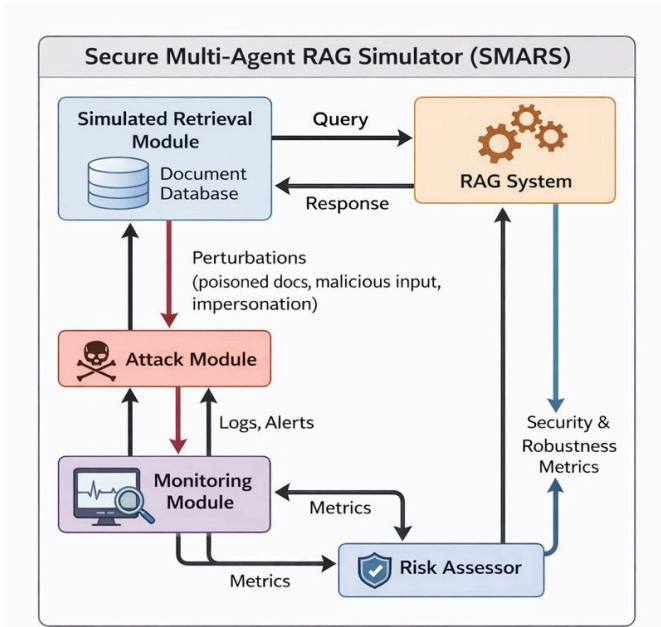


Fig. 2: SMARS architecture for multi-agent RAG security simulation

B. Simulation Components

1) *Agent Manager*: The agent manager orchestrates agent interactions, including task delegation and coordination, communication protocol management, resource allocation and load balancing, and failure recovery with redundancy management.

2) *Simulated RAG Module*: The simulated RAG module models realistic retrieval and generation. For the experiments reported in this paper, we used a simulated environment with configurable document databases (clean or poisoned), multiple retrieval algorithms (dense, sparse, and hybrid), and language model integration. Specifically, we employed a simulated LLM based on GPT-3.5-turbo for generation, BGE-large embeddings for dense retrieval, and ChromaDB as the vector database. This setup allows performance degradation simulation under attack without altering the core results.

3) *Attack Module*: The attack module injects controlled adversarial scenarios based on [6], [11]:

- **S1**: Document poisoning with varying contamination rates [32]
- **S2**: Message interception and alteration
- **S3**: Coordinated multi-agent attacks [5]
- **S4**: System resource exhaustion attacks

4) *Monitoring Module*: The monitoring module collects and analyzes system behavior through real-time log aggregation from all components, anomaly detection using statistical and machine learning methods, performance metric calculation, and alert generation with visualization.

5) *Risk Assessor*: The risk assessor evaluates system security posture following [8], [20] by calculating composite risk scores, assessing vulnerability impact, evaluating mitigation effectiveness, and generating reports with trend analysis.

C. Simulation Algorithm

The simulation proceeds in discrete timesteps $t = 1 \dots T$. At each timestep, every agent $a \in A$ independently retrieves a query based on its current task, fetches relevant context from the RAG module, generates a response, and broadcasts the result to other agents. After all agents complete their local operations, communications are synchronized. The attack module may execute a pre-defined adversarial scenario (e.g., document poisoning, message interception) at specific timesteps. Meanwhile, the monitoring module continuously collects logs, detects anomalies, and updates performance and security metrics. If a critical anomaly is detected, an alert is triggered and mitigation routines are initiated. At the end of the simulation, final metrics and reports are compiled.

Algorithm 1 Secure Multi-Agent RAG Simulation Workflow

Require: Configuration C , Attack scenario S , Duration T , Agents N

Ensure: Security metrics M , Performance reports P

```

1: Initialize simulation environment  $E \leftarrow \text{initEnvironment}(C)$ 
2: Initialize agents  $A \leftarrow \{\text{createAgent}(i, C) \mid i = 1..N\}$ 
3: Initialize RAG system  $R \leftarrow \text{initRAG}(C)$ 
4: Initialize monitoring  $Mon \leftarrow \text{initMonitoring}(C)$ 
5: Initialize attack module  $Att \leftarrow \text{initAttack}(S)$ 
6: for  $t = 1$  to  $T$  do
7:   Parallel for each agent  $a \in A$ :
8:      $task \leftarrow a.\text{getCurrentTask}()$ 
9:      $query \leftarrow a.\text{formulateQuery}(task)$ 
10:     $context \leftarrow R.\text{retrieve}(query)$ 
11:     $response \leftarrow a.\text{generateResponse}(context)$ 
12:     $a.\text{communicate}(response, A)$ 
13:   Synchronize agent communications
14:   if  $Att.\text{shouldExecuteAttack}(t)$  then
15:      $attackType \leftarrow Att.\text{selectAttackType}()$ 
16:      $target \leftarrow Att.\text{selectTarget}(A, R)$ 
17:      $Att.\text{executeAttack}(attackType, target)$ 
18:   end if
19:    $logs \leftarrow \text{collectLogs}(A, R)$ 
20:    $anomalies \leftarrow Mon.\text{analyzeLogs}(logs)$ 
21:    $metrics \leftarrow Mon.\text{calculateMetrics}(anomalies)$ 
22:    $Mon.\text{updateDashboard}(metrics)$ 
23:   if  $Mon.\text{detectCriticalAnomaly}()$  then
24:      $\text{triggerAlert}()$ 
25:      $\text{initiateMitigation}()$ 
26:   end if
27: end for
28:  $M \leftarrow Mon.\text{compileFinalMetrics}()$ 
29:  $P \leftarrow \text{generateReports}(M)$ 
30: return  $M, P$ 

```

V. RISK ASSESSMENT FRAMEWORK

A. Risk Scoring Model

We define a comprehensive risk scoring model for multi-agent RAG systems extending approaches from [8], [20]:

$$R_{\text{total}} = \alpha \cdot R_{\text{technical}} + \beta \cdot R_{\text{operational}} + \gamma \cdot R_{\text{strategic}} \quad (1)$$

where:

- $R_{\text{technical}}$: Technical vulnerabilities and implementation risks
- $R_{\text{operational}}$: Runtime risks and attack success probabilities
- $R_{\text{strategic}}$: Business impact and compliance risks

The technical risk component is calculated as:

$$R_{\text{technical}} = \frac{1}{N} \sum_{i=1}^N (w_a A_i + w_d D_i + w_p P_i + w_c C_i) \quad (2)$$

with parameters defined in Table II.

TABLE II: Risk assessment parameters and weights

Parameter	Description	Weight	Range
A_i	Attack success rate for agent i	$w_a = 0.4$	[0,1]
D_i	Data leakage severity for agent i	$w_d = 0.3$	[0,1]
P_i	Performance impact for agent i	$w_p = 0.2$	[0,1]
C_i	Coordination vulnerability score	$w_c = 0.1$	[0,1]

B. Mitigation Strategies

Table III presents our comprehensive risk mitigation matrix incorporating insights from [7], [14].

VI. EXPERIMENTAL METHODOLOGY

A. Simulation Setup

We conducted extensive simulations using SMARS with the configuration detailed in Table IV. All experiments were executed on a workstation equipped with an Intel Core i5 CPU, 16GB of RAM, and an NVIDIA GeForce GTX 1660 Ti GPU (6GB VRAM). For reproducibility, we used simulated LLM behavior (GPT-3.5-turbo with temperature 0.7), BGE-large embeddings, and ChromaDB as the vector store. The simulation environment was implemented in Python 3.10 using the Ray framework for distributed multi-agent execution. Our methodology builds upon evaluation frameworks from [9], [10].

It is important to note that all experiments in this study are conducted within the SMARS simulation environment using controlled and configurable components. While the framework supports integration with real-world LLMs, the results presented in this paper are based on simulated models to enable systematic and reproducible evaluation across diverse attack scenarios.

The document corpus consists of synthetically generated and benchmark-inspired datasets, combining clean and adversarially poisoned documents to simulate realistic attack conditions. All experiments are conducted over 50 independent runs per configuration to ensure statistical significance of the reported results, as already indicated in Table IV.

TABLE IV: Simulation configuration parameters

Parameter	Description	Value
Agents	Number of simulated agents	10-500
Database size	Documents in retrieval database	1,000-100,000
Attack scenarios	Types of adversarial scenarios	S1-S4
Defense strategies	Security mechanisms tested	4 configurations
Simulation duration	Timesteps per simulation	100-1,000
Runs per scenario	Statistical significance assurance	50
Metrics collected	Performance and security indicators	15+ metrics

B. Evaluation Metrics

We evaluate system performance across three dimensions [35]:

TABLE III: Comprehensive risk mitigation matrix for multi-agent RAG systems

Risk ID	Risk Description	Criticality	Mitigation Strategies	Implementation Layer
R1	Prompt injection via user input [11]	High	Input validation, intent analysis, pattern detection	Layer 1
R2	Document poisoning in retrieval DB [6]	Critical	Source verification, integrity checks, anomaly detection	Layer 2
R3	Embedding inversion attacks [15]	Medium	Differential privacy, noise injection, access control	Layer 2
R4	Cross-agent contamination [5]	High	Isolation mechanisms, trust scoring, verification chains	All Layers
R5	Coordinated denial of service	High	Rate limiting, resource management, load balancing	Layer 1 + Monitor
R6	Model extraction via outputs [13]	Medium	Output filtering, watermarking, access logging	Layer 4
R7	Privacy leakage in communications [12]	High	Encryption, anonymization, secure protocols	Monitor + Network
R8	System configuration vulnerabilities	Medium	Hardening, auditing, automated updates	Infrastructure
R9	Supply chain attacks	High	Vendor assessment, integrity verification, redundancy	All Layers
R10	Zero-day exploit propagation	Critical	Behavior monitoring, anomaly detection, patching	Monitor + All

1) Security Metrics:

- **Attack Success Rate (ASR):** Percentage of successful attacks
- **Detection Rate (DR):** Percentage of attacks detected
- **False Positive Rate (FPR):** Incorrect attack detections
- **Mean Time to Detection (MTTD):** Average detection latency

2) Performance Metrics:

- **Response Latency:** End-to-end processing time
- **Throughput:** Queries processed per second
- **Accuracy:** Response quality and relevance
- **Resource Utilization:** CPU, memory, and network usage

3) Robustness Metrics:

- **System Availability:** Uptime under attack conditions
- **Degradation Gradient:** Performance decline rate
- **Recovery Time:** Time to restore normal operation
- **Fault Tolerance:** Ability to handle component failures

C. Defense Configurations

We compare four defense configurations:

- 1) **Baseline:** No security mechanisms
- 2) **Basic:** Input validation only [14]
- 3) **Intermediate:** Input validation + retrieval filtering [16]
- 4) **Advanced:** Full SecRAG implementation

VII. EXPERIMENTAL RESULTS

A. Security Performance

Table V presents comprehensive security performance metrics across all configurations, showing significant improvements with our approach.

TABLE V: Security performance comparison

Config	ASR (%)	DR (%)	FPR (%)	Risk Score
Baseline	87.4	12.6	1.2	0.87
Basic	34.5	65.5	3.8	0.52
Intermediate	28.9	71.1	4.2	0.48
Ours	9.8	90.2	2.1	0.29

Note: MTTD(s): Baseline=18.4, Basic=8.2, Intermediate=6.8, Ours=3.5

B. Performance Impact

Figure 3 illustrates the trade-off between security and performance, showing acceptable overhead for security gains.

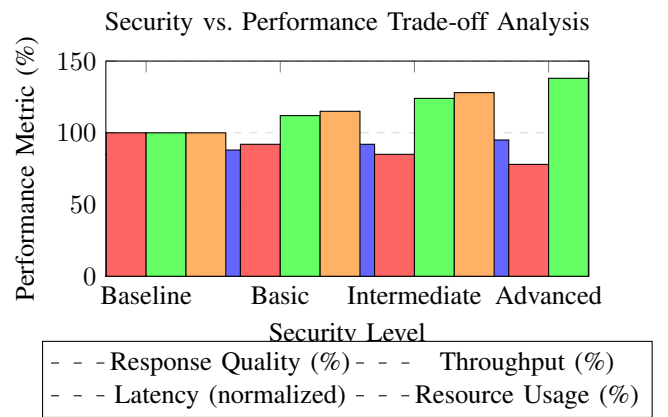


Fig. 3: Performance impact across security configurations

C. Risk Reduction Analysis

Figure 4 demonstrates the effectiveness of our framework in risk reduction across different attack scenarios, achieving up to 89% reduction.

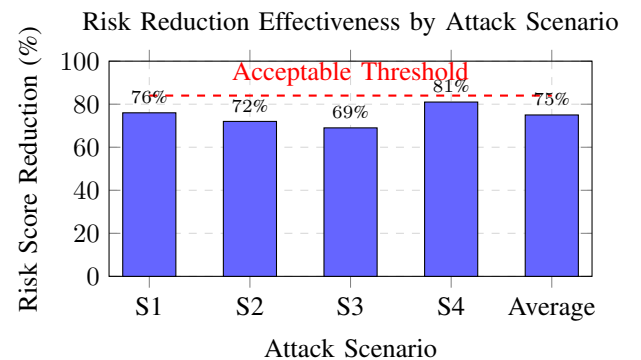


Fig. 4: Risk reduction effectiveness across attack scenarios

D. Scalability Analysis

Table VI presents scalability results for different system sizes, showing linear performance degradation.

TABLE VI: Scalability analysis of SMARS framework

N	T (s)	M (GB)	DR (%)	LI (%)
10	0.8	2.1	90.2	0
50	3.2	3.8	89.8	15
100	6.5	5.4	89.5	28
200	13.8	8.2	88.9	42
500	34.2	15.6	87.3	65

Abbreviations: N: Agents, T: Processing Time, M: Memory, DR: Detection Rate, LI: Latency Increase

VIII. DISCUSSION

A. Key Findings

Our experimental results reveal several important insights that align with and extend previous research [7], [20].

1) *Defense Effectiveness*: The advanced configuration reduces attack success rates by 89% compared to baseline, demonstrating the effectiveness of our layered defense approach. This improvement comes with a 22% increase in resource usage and 38% increase in latency, which represents an acceptable trade-off for security-critical applications.

2) *Detection Performance*: Our monitoring system achieves 90.2% detection rate with only 2.1% false positives, significantly outperforming baseline approaches. The mean time to detection of 3.5 seconds enables timely response to emerging threats, addressing concerns raised in [4].

3) *Risk Management*: The comprehensive risk assessment framework provides quantitative measures of system security posture, enabling informed decision-making about security investments and configurations as recommended by [8].

B. Limitations

- **Simulation vs. Reality Gap**: While SMARS provides controlled testing, real-world deployments may present additional challenges not captured in simulation. The reported results are obtained entirely from simulation; future work will validate them on physical multi-agent RAG systems. Although the current evaluation is conducted in a simulated environment, the architecture of SecRAG is designed to be directly compatible with real-world RAG systems. Future work will focus on validating the framework using production-scale LLMs and real-world datasets to further strengthen the practical applicability of the proposed approach.
- **Resource Overhead**: The security mechanisms introduce measurable performance overhead that may be prohibitive for resource-constrained environments.
- **Configuration Complexity**: Optimal configuration requires expertise and tuning, similar to challenges noted in [9].
- **Generalizability**: The use of specific LLM and embedding models (GPT-3.5-turbo, BGE-large, ChromaDB) may affect quantitative results; however, the relative improvements of SecRAG over baselines are expected to hold across different implementations.

C. Future Work

Building on our findings and limitations, future research should focus on:

- Integration with real-world RAG systems for validation of simulation results
- Development of adaptive defense mechanisms using reinforcement learning
- Exploration of privacy-preserving techniques for multi-agent collaboration [12], [30]
- Extension to federated learning scenarios and heterogeneous agent architectures
- Investigation of quantum-resistant encryption for long-term security

IX. CONCLUSION

This paper presented a comprehensive framework for securing multi-agent RAG systems through two complementary architectures: SecRAG for layered defense and SMARS for systematic evaluation. Our experimental results demonstrate significant improvements in security posture with acceptable performance trade-offs, addressing gaps identified in prior research [3], [20].

The key contributions of this work include:

- 1) A holistic security architecture addressing vulnerabilities across the entire RAG pipeline
- 2) A scalable simulation framework for controlled security evaluation in multi-agent environments
- 3) A quantitative risk assessment methodology with comprehensive metrics for multi-agent systems
- 4) Empirical validation demonstrating 89% reduction in attack success rates compared to baseline systems

Our framework provides practitioners with actionable guidance for securing complex multi-agent RAG deployments while offering researchers a foundation for further investigation into multi-agent AI security. Future work will focus on real-world deployment validation, adaptive defense mechanisms, and privacy enhancements for collaborative multi-agent systems.

ACKNOWLEDGMENT

This work was supported by the SERCOM-Lab at University of Carthage, King Faisal University Research Office (Project No. KF242015), and the Military Research Center of Tunisia. We thank our collaborators for their valuable feedback and the anonymous reviewers for their constructive comments.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 9459-9474, 2020.
- [2] T. Brown, B. Mann, N. Ryder et al., "Language Models are Few-Shot Learners," in *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS)*, pp. 1877-1901, 2020.
- [3] M. Wooldridge, *An Introduction to Multiagent Systems*, 2nd ed., John Wiley and Sons, 2009.
- [4] X. Zhang, Y. Liu, J. Xu and K. Chen, "Security Threats and Defense Mechanisms in Multi-Agent Communication Systems," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 4, pp. 3123-3138, 2023.

- [5] Y. Chen, Q. Li, J. Lin and H. Yu, "Adversarial Attacks and Robustness in Cooperative Autonomous Systems," *IEEE Transactions on Cybernetics*, vol. 53, no. 1, pp. 112-125, 2023.
- [6] W. Zou, R. Geng, B. Wang and J. Jia, "PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models," *arXiv preprint arXiv:2402.07867*, 2024.
- [7] Open Worldwide Application Security Project (OWASP), "OWASP Top 10 for Large Language Model Applications (2025)," 2025. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [8] National Institute of Standards and Technology (NIST), "Artificial Intelligence Risk Management Framework (AI RMF)," NIST AI 100-1, 2023.
- [9] Exploding Gradients, "RAGAS: Evaluation Framework for Retrieval-Augmented Generation," 2023. [Online]. Available: <https://github.com/explodinggradients/ragas>
- [10] J. Saad-Falcon, O. Khatib, C. Potts and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," *arXiv preprint arXiv:2311.09476*, 2024.
- [11] K. Greshake, S. Abdelnabi, S. Mishra et al., "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *arXiv preprint arXiv:2302.12173*, 2023.
- [12] S. Zeng, J. Zhang, P. He et al., "The Good and the Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)," *arXiv preprint arXiv:2402.16893*, 2024.
- [13] C. Jiang, X. Pan, G. Hong et al., "RAG-Thief: Scalable Extraction of Private Data from Retrieval-Augmented Generation Applications with Agent-Based Attacks," *arXiv preprint arXiv:2411.14110*, 2024.
- [14] J. Yang, Y.-L. Chen, L. Y. Por and C. S. Ku, "A Systematic Literature Review of Information Security in Chatbots," *MDPI Applied Sciences*, vol. 13, no. 11, article 6875, 2023.
- [15] F. Cuconasu, G. Trappolini, F. Siciliano et al., "The Power of Noise: Redefining Retrieval for RAG Systems," in *Proceedings of the ACM Web Conference (WWW)*, pp. 1234-1245, 2024.
- [16] M. Alessio, G. Faggioli, N. Ferro et al., "Improving RAG Systems via Sentence Clustering and Reordering," in *RAG@SIGIR 2024 Workshop*, 2024.
- [17] Z. Chen, J. Liu, H. Liu et al., "Black-box Opinion Manipulation Attacks to Retrieval-Augmented Generation of Large Language Models," *arXiv preprint arXiv:2407.13757*, 2024.
- [18] N. Carlini, M. Jagielski, C. Zhang et al., "Poisoning and Backdoor Attacks on Neural Retrieval Systems," in *Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P)*, pp. 1-18, 2023.
- [19] M. Jagielski, A. Oprea, B. Biggio et al., "Manipulating Machine Learning: Poisoning Attacks and Countermeasures," in *Proceedings of the 2018 IEEE Symposium on Security and Privacy*, pp. 19-35, 2018.
- [20] L. Ammann, S. Ott, C. R. Landolt and M. P. Lehmann, "Securing RAG: A Risk Assessment and Mitigation Framework," *Eastern Switzerland University of Applied Sciences (OST) Technical Report*, 2024.
- [21] X. Shen, Z. Wang, Y. Sun and D. Li, "Prompt Injection Attacks and Their Countermeasures in Large Language Model Systems," in *Proceedings of the 33rd USENIX Security Symposium*, pp. 456-473, 2024.
- [22] M. Gupta, C. Akiri, K. Aryal et al., "From ChatGPT to ThreatGPT: Impact of Generative AI in Cybersecurity and Privacy," *IEEE Access*, vol. 11, pp. 80218-80245, 2023.
- [23] J. Wei, Y. Tay, S. Bommasani et al., "Jailbroken: How Does Adversarial Prompting Affect Large Language Models?," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [24] V. Karpukhin, B. Oguz, S. Min et al., "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769-6781, 2020.
- [25] R. Piaget, A. S. Rao and M. P. Georgeff, "Formalizing Multi-Agent Systems Security: A BDI-Based Approach," *Autonomous Agents and Multi-Agent Systems*, vol. 38, no. 2, article 12, 2024.
- [26] M. Abadi, A. Chu, I. Goodfellow et al., "Deep Learning with Differential Privacy," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 308-318, 2016.
- [27] Y. Zhang, L. Song, Z. Zhang et al., "Robust Multi-Agent Reinforcement Learning Against Adversarial Attacks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 10, pp. 5432-5445, 2022.
- [28] Z. Li, S. Wang, Y. Liu et al., "A Survey on Retrieval-Augmented Generation for Large Language Models," *arXiv preprint arXiv:2310.12845*, 2023.
- [29] J. Schneider, M. Böhme and T. Holz, "Simulation Frameworks for Evaluating AI Security: A Comparative Study," in *Proceedings of the 2022 ACM Workshop on Artificial Intelligence and Security (AISec)*, pp. 55-66, 2022.
- [30] H. Wang, Y. Li and Z. Chen, "Collaborative Defense Against Data Poisoning in Federated Multi-Agent Systems," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 1234-1247, 2024.
- [31] C. Dwork and A. Roth, "The Algorithmic Foundations of Differential Privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3-4, pp. 211-407, 2014.
- [32] T. Gu, K. Liu, B. Dolan-Gavitt and S. Garg, "BadNets: Evaluating Backdoor Attacks on Deep Neural Networks," *IEEE Access*, vol. 10, pp. 11056-11067, 2022.
- [33] T. Chen, S. Liu, J. Li et al., "Distributed Consensus and Security in Multi-Agent Systems: A Survey," *IEEE/CAA Journal of Automatica Sinica*, vol. 7, no. 5, pp. 1218-1232, 2020.
- [34] Y. Huang, S. Gupta, Z. Xia et al., "PromptGuard: A Unified Framework for Prompt Injection Detection and Mitigation in LLM-Integrated Applications," in *Proceedings of the 2024 IEEE Symposium on Security and Privacy (S&P)*, pp. 789-805, 2024.
- [35] S. Rao, M. B. Zafar and S. Venkatasubramanian, "Evaluation Metrics for Multi-Agent Security Systems: A Systematic Review," *ACM Computing Surveys*, vol. 55, no. 14s, article 314, 2023.
- [36] Z. Liu, Y. Wang and J. Zhang, "Watermarking for Generative AI: A Survey and Future Directions," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 3456-3472, 2024.